

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Обработка изображений в интеллектуальных системах»
Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:
Студент 4 курса
Группы ИИ-25
Заседатель Н.С.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Задание:

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
6	MNIST	28X28	Adam

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from PIL import Image, ImageOps
import matplotlib.pyplot as plt
import os

batchsize = 64
epochs = 5
Lr = 0.001

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Устройство:", device)

transform = transforms.ToTensor()

train_data = datasets.MNIST(
    "./data",
    train=True,
    download=True,
    transform=transform
)

test_data = datasets.MNIST(
    "./data",
```

```

        train=False,
        download=True,
        transform=transform
    )

train_loader = DataLoader(
    train_data,
    batch_size=batchsize,
    shuffle=True
)

test_loader = DataLoader(
    test_data,
    batch_size=batchsize,
    shuffle=False
)

print("Обучающая выборка:", len(train_data))
print("Тестовая выборка:", len(test_data))

class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x

model = CNN().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=Lr)

losses = []
accuracies = []

print("\nОбучение:")

for epoch in range(epochs):

    model.train()
    total_loss = 0

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

```

```

optimizer.zero_grad()

output = model(images)
loss = criterion(output, labels)

loss.backward()
optimizer.step()

total_loss += loss.item()

loss_value = total_loss / len(train_loader)
losses.append(loss_value)

model.eval()

correct = 0
total = 0

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        output = model(images)
        prediction = output.argmax(dim=1)

        correct += (prediction == labels).sum().item()
        total += labels.size(0)

accuracy = correct / total * 100
accuracies.append(accuracy)

print(
    f"Эпоха {epoch + 1}/{epochs} | "
    f"Ошибка: {loss_value:.4f} | "
    f"Точность: {accuracy:.2f}%"
)

print(f"\nИтоговая точность: {accuracies[-1]:.2f}%")

plt.plot(range(1, epochs + 1), losses, marker="o")
plt.xlabel("Эпоха")
plt.ylabel("Ошибка")
plt.title("Изменение ошибки при обучении")
plt.grid()
plt.show()

model.eval()

plt.figure(figsize=(9, 6))

for i in range(6):

    image, label = test_data[
        torch.randint(len(test_data), (1,)).item()
    ]

    with torch.no_grad():
        output = model(image.unsqueeze(0).to(device))
        prediction = output.argmax(dim=1).item()

    plt.subplot(2, 3, i + 1)

```

```

plt.imshow(image.squeeze(), cmap="gray")
plt.title(f"Правильная: {label}, сеть: {prediction}")
plt.axis("off")

plt.tight_layout()
plt.show()

folder = "my_digits"

if os.path.exists(folder):

    files = os.listdir(folder)

    for file in files:

        if not file.lower().endswith((".png", ".jpg", ".jpeg")):
            continue

        path = os.path.join(folder, file)

        image = Image.open(path).convert("L")

        if image.getpixel((0, 0)) > 128:
            image = ImageOps.invert(image)

        box = image.getbbox()

        if box:
            image = image.crop(box)

        size = max(image.size)
        new_image = Image.new("L", (size, size), 0)

        x = (size - image.width) // 2
        y = (size - image.height) // 2

        new_image.paste(image, (x, y))

        image = new_image.resize((28, 28))

        image_tensor = transforms.ToTensor()(image)
        image_tensor = image_tensor.unsqueeze(0).to(device)

        with torch.no_grad():
            output = model(image_tensor)
            prediction = output.argmax(dim=1).item()

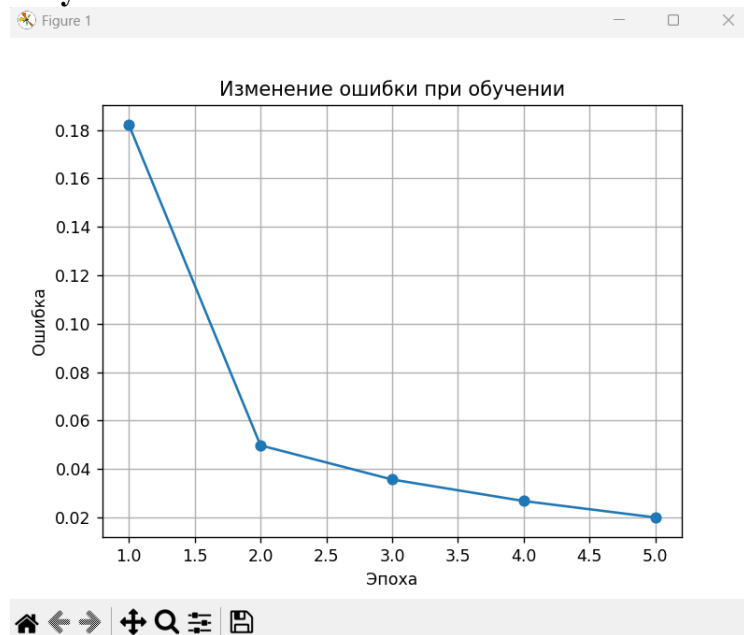
        print(f"{file} - {prediction}")

        plt.figure(figsize=(3, 3))
        plt.imshow(image, cmap="gray")
        plt.title(f"{file}: сеть думает это {prediction}")
        plt.axis("off")
        plt.show()

else:
    print("\nПапка my_digits не найдена")

```

Результат:



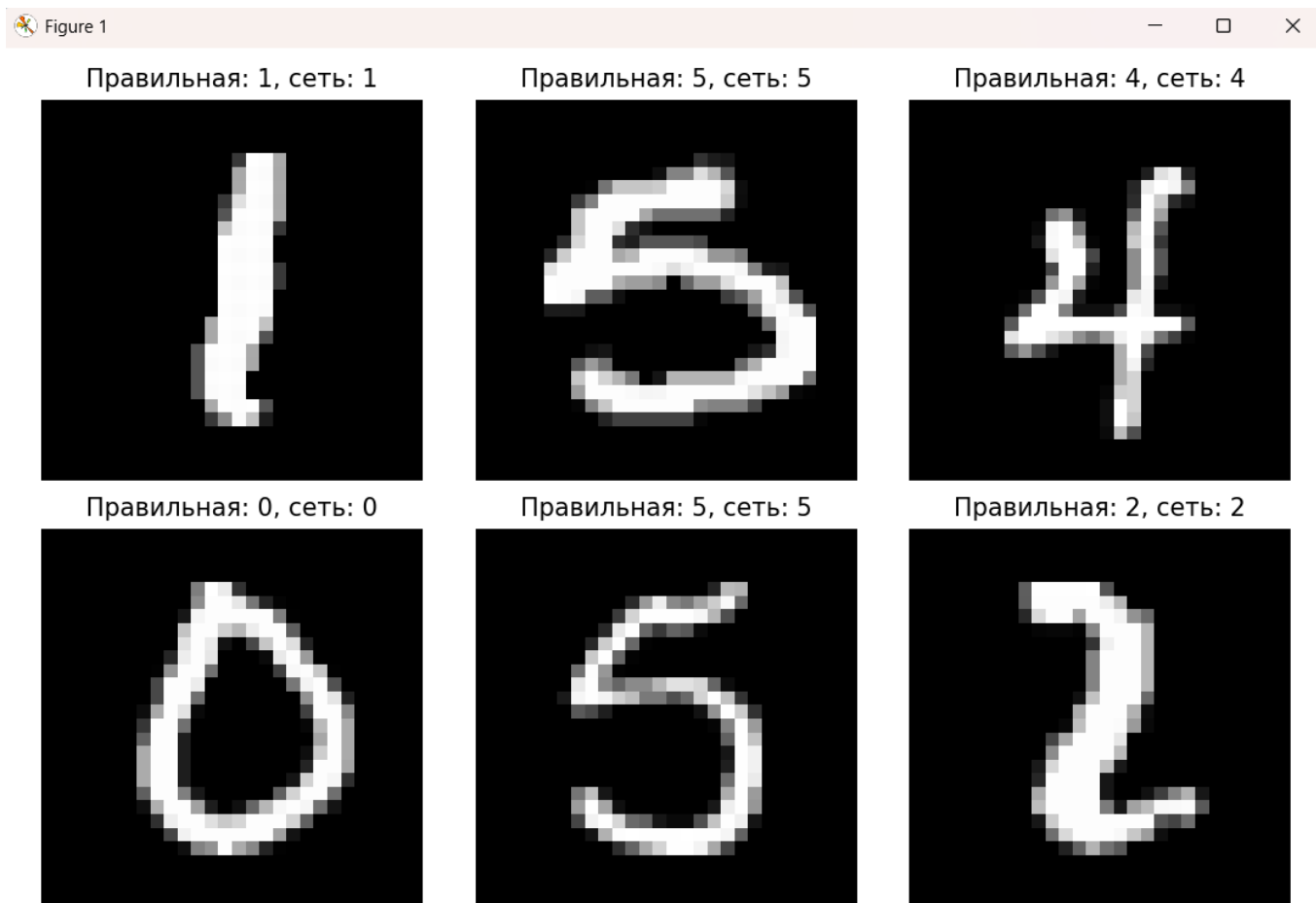
Устройство: cpu
Обучающая выборка: 60000
Тестовая выборка: 10000

Обучение:

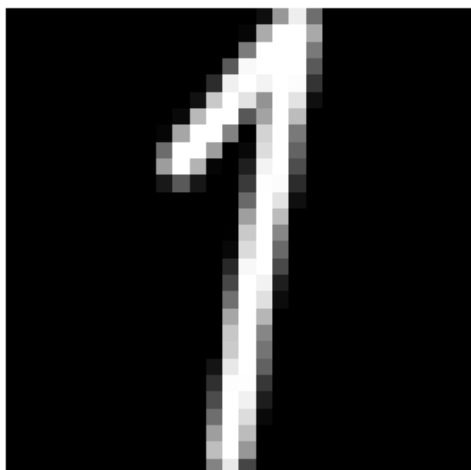
Эпоха 1/5	Ошибка: 0.1822	Точность: 98.14%
Эпоха 2/5	Ошибка: 0.0497	Точность: 98.91%
Эпоха 3/5	Ошибка: 0.0356	Точность: 98.85%
Эпоха 4/5	Ошибка: 0.0267	Точность: 99.05%
Эпоха 5/5	Ошибка: 0.0200	Точность: 98.98%

Итоговая точность: 98.98%

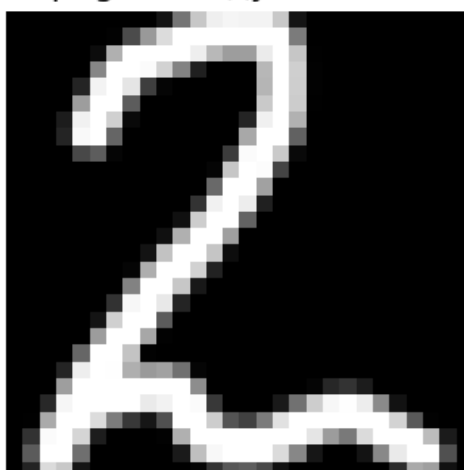
1.png - 1
2.png - 2
3.png - 3
4.png - 4
5.png - 5
6.png - 6
8.png - 8



1.png: сеть думает это 1



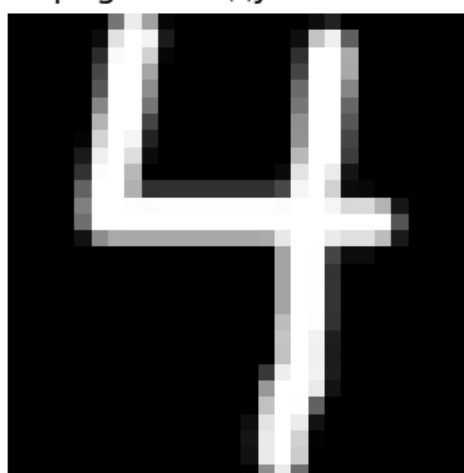
2.png: сеть думает это 2



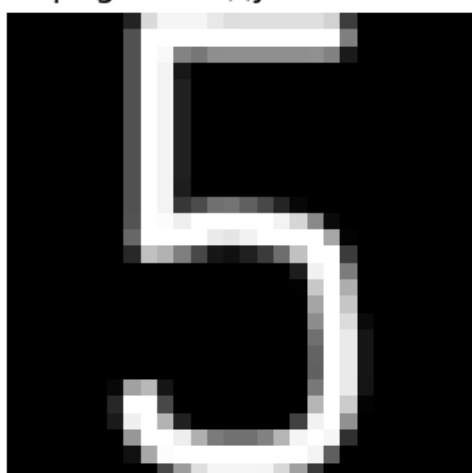
3.png: сеть думает это 3



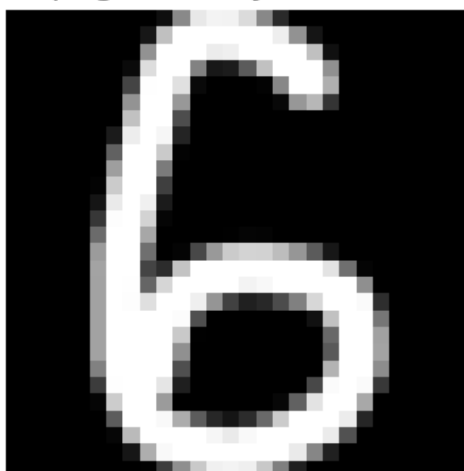
4.png: сеть думает это 4



5.png: сеть думает это 5



6.png: сеть думает это 6



Вывод: Научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения